CRUCIBLE · SYSTEM DESIGN DOCUMENT

Multi-Agent Code Review & Autonomous Arbitration Architecture

Course: Flexi-CA3 | Orchestrator-Worker with Blackboard Pattern

1. ARCHITECTURE OVERVIEW & COMMUNICATION PATTERN

Crucible implements a hybrid Blackboard + Dynamic Peer-Critique + Hierarchical Arbitration multi-agent topology orchestrated using LangGraph state machines. All agent interactions occur asynchronously through an append-only shared blackboard state (CrucibleState) rather than point-to-point hardcoded calls. When specialists produce conflicting findings (e.g. Logic Reviewer grading code correct while Security Auditor flags an unvalidated SQL query), the Orchestrator initiates a targeted critique round. If consensus remains unreached, the Orchestrator executes binding LLM arbitration with explicit written rationales.

2. AGENT SPECIFICATIONS & SPECIALIST TOOLS

Agent	LLM Engine	Static Tools	Core Responsibility
Orchestrator	Gemini 3.5 Flash	LangGraph, Blackboard	Triage, worker dispatch, conflict detection, binding arbitration & synthesis.
Logic Reviewer	Gemini 3.5 Flash	AST Parser, Radon CC	Algorithmic correctness, off-by-one errors, edge cases, cyclomatic complexity.
Security Auditor	Gemini 3.5 Flash	Bandit SAST, Secret Scanner	SQL injection (CWE-89), hardcoded secrets (CWE-798), CVE/CWE tagging.
Style & Doc Agent	Gemini 3.5 Flash	pycodestyle, pydocstyle	PEP 8/ESLint formatting, naming conventions, docstring completeness (PEP 257).

3. AUTONOMOUS ROUTING & STATE CONVERGENCE

Crucible replaces rigid sequential execution with dynamic LLM-driven dispatching:

- Non-Hardcoded Routing: The Orchestrator inspects incoming code characteristics and selects which specialist agents to run and what specific focus areas to assign.
- Termination Safeguard: A hard limit (budget_rounds = 3) prevents runaway cycles, ensuring guaranteed convergence.
- Traceability: Every state transition emits an immutable TraceEvent payload with timestamps and decision rationales.

4. ROBUSTNESS & FAULT TOLERANCE (CATEGORY C2 COMPLIANCE)

Crucible incorporates an isolated tool execution harness (tools/tool_base.py) with exception containment:

- Simulated Fault Injection: If a static tool fails (e.g. Bandit crashing in Scenario D), Crucible catches the failure without crashing the pipeline.
- Autonomous Re-Planning: The affected agent immediately emits a 'fallback' trace event and seamlessly transitions to pure LLM deep-reasoning to complete the review.
- Language Agility: When reviewing non-Python code (JavaScript, Go), static tools automatically record a fallback and the LLM agents autonomously evaluate language-specific vulnerabilities, complexity, and idiomatic conventions.

5. TECHNOLOGY STACK

- Multi-Agent Framework:	LangGraph 1.2+ (StateGraph, StateMachine, MemorySaver)
- LLM Foundation:	Google Gemini 3.5 Flash via google-generativeai SDK
- Web API Server:	FastAPI (Python 3.11), Uvicorn ASGI, Server-Sent Events (SSE)
- Static Analysis Suite:	Bandit (SAST), Radon (Cyclomatic Complexity), pycodestyle, pydocstyle
- Client Dashboard:	Responsive Single Page Architecture, CSS Glassmorphism, Vanilla JS SSE client
- Containerization & Cloud:	Docker (python:3.11-slim) & Render.com automated PaaS deployment

6. AUTO-FIX REMEDIATION & DEPLOYMENT

In addition to detection, Crucible provides autonomous patch remediation (/autofix endpoint). Detected vulnerabilities (such as raw SQL string concatenation CWE-89) are patched using parameterized queries and prepared statements, while cyclomatic complexity is flattened into clean, modular helper structures. The web application is fully containerized using a production Dockerfile and deployed on Render.com with continuous health probe monitoring (/health).